SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	**DSA03**	**Course Title:**	**Natural Language Processing**
CO Assessed:	**CO1 – Imitation (L1)**	**Assessment:**	**Assessment Tool 3 – Coding Challenge**
Weightage:	**30% of CIA**		
CO5 Statement:	**Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)**		
Student Name:	**A. Abhishek Reddy**	**Reg. No.:**	**192424192**
Section / Batch:	**AI&DS (2024)**	**Date:**	**15/10/26**

Code of Conduct

I* A. Abhishek Reddy (192424192) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Abhishek Reddy

Instructions

- **Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.**

- **Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.**
- **Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.**
- **Duration: 40 minutes.**

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- **Accept the DFA description**
  - **States**
  - **Input alphabet**
  - **Transition table**
  - **Initial state**
  - **Final state(s)**
- **Accept multiple input strings.**
- **Simulate state transitions.**
- **Display whether the string is Accepted or Rejected.**

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

q0 → q1 → q0 → q1 → q2

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Problem Understanding	**20%**	**Clearly understands problem constraints, edge cases, and competitive requirements**	**Understands problem with minor gaps in constraints**	**Partial understanding; misses key constraints**	**Misinterprets problem or constraints**
Algorithm	**25%**	**Selects optimal algorithm with best time and space complexity for competitive constraints**	**Uses efficient algorithm with minor scope for improvement**	**Uses basic/inefficient approach**	**No clear algorithmic approach**
Code Implementation	**20%**	**Writes correct, efficient, and clean code that passes all constraints**	**Code works with minor errors or inefficiencies**	**Partially correct code; fails for some cases**	**Code does not execute or gives incorrect results**
Competitive Performance	**20%**	**Solves problem within time limits and handles large inputs effectively**	**Solves within acceptable time with minor delays**	**Struggles with time limits or larger inputs**	**Unable to solve within time constraints**
Test Case Handling	**15%**	**Handles all test cases including edge and hidden cases**	**Handles most test cases**	**Misses important edge cases**	**Fails on multiple test cases**

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

1.A.Aim: To validate an email address, password, and mobile number using Python regular expressions.

CODE:

```
import re
email = input("Enter email: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
if re.fullmatch(password_pattern, password):
```

```
    print("Strong Password")
else:
    print("Weak Password")
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

OUTPUT:

```
Enter email: student123@saveetha.edu
Enter password: Student@123
Enter mobile number: 9876543210
Valid Email
Strong Password
Valid Mobile Number
```

RESULT: The email, password, and mobile number validator was successfully implemented using Python Regular Expressions. It displays valid or invalid messages based on the given rules.

Section 2: Design a Finite State Automata (FSA) Simulator

1. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- **Accept the DFA description**
  - **States**
  - **Input alphabet**
  - **Transition table**
  - **Initial state**
  - **Final state(s)**
- **Accept multiple input strings.**
- **Simulate state transitions.**
- **Display whether the string is Accepted or Rejected.**

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

q0 → q1 → q0 → q1 → q2

Accepted

2.A.Aim: To implement a Deterministic Finite Automaton (DFA) that accepts strings ending with ab.

CODE:

```
states = input("Enter states: ").split()
alphabet = input("Enter input alphabet: ").split()
transition = {}
for state in states:
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(
            f"Transition for {state} with {symbol}: "
        )
        transition[state][symbol] = next_state
initial = input("Enter initial state: ")
final_states = input("Enter final states: ").split()
n = int(input("Enter number of strings: "))
for i in range(n):
    string = input("Enter string: ")
    current = initial
    path = [current]
    valid = True
```

```
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        current = transition[current][symbol]
        path.append(current)
    print("Transition Path:", " -> ".join(path))
    if valid and current in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

OUTPUT:

```
Enter states: q0 q1 q2
Enter input alphabet:  a b
Transition for q0 with a: q1
Transition for q0 with b: q0
Transition for q1 with a:  q1
Transition for q1 with b:  q2
Transition for q2 with a: q1
Transition for q2 with b: q0
Enter initial state:  q0
Enter final states: q2
Enter number of strings: 2
Enter string: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string:  aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Rejected
```

RESULT: The DFA simulator was successfully implemented in Python. It displays the transition path and determines whether the input strings are accepted or rejected

Section 3: Smart Pattern Matching Engine using Regular Expressions

1. **Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffi

Expected Output

Display all matching patterns based on user selection.

3.A.Aim: To search text for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions using Python regular expressions.

CODE:

```
import re

text = input("Enter text: ")

print("\n1. Search Word")

print("2. Search Date")
```

```
print("3. Search Phone Number")
print("4. Search Hashtag")
print("5. Search Mention")
print("6. Search Prefix")
print("7. Search Suffix")
choice = input("Enter your choice: ")
if choice == "1":
    word = input("Enter word: ")
    pattern = r'\b' + re.escape(word) + r'\b'
    result = re.findall(pattern, text, re.I)
elif choice == "2":
    pattern = r'\b\d{2}/\d{2}/\d{4}\b'
    result = re.findall(pattern, text)
elif choice == "3":
    pattern = r'\b[6-9]\d{9}\b'
    result = re.findall(pattern, text)
elif choice == "4":
    pattern = r'(?<!\w)#[A-Za-z0-9_]+'
    result = re.findall(pattern, text)
elif choice == "5":
    pattern = r'(?<!\w)@[A-Za-z0-9_]+'
    result = re.findall(pattern, text)
elif choice == "6":
    word = input("Enter prefix: ")
    pattern = r'\b' + re.escape(word) + r'\w*'
    result = re.findall(pattern, text, re.I)
elif choice == "7":
    word = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(word) + r'\b'
    result = re.findall(pattern, text, re.I)
```

```
else:
    result = None
    print("Invalid choice")
if result is not None:
    if result:
        print("\nMatching Patterns:")
        for item in result:
            print(item)
        print("Total Matches:", len(result))
    else:
        print("No matching patterns found")
```

OUTPUT:

```
Enter text: Meeting on 12/09/2026. Call 9876543210 #NLP @OpenAI natural language processing

1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
Enter your choice: 2

Matching Patterns:
12/09/2026
Total Matches: 1
```

RESULT: The pattern matching engine was successfully implemented using Python Regular Expressions. It searches for words, dates, phone numbers, hashtags, mentions, prefixes, and suffixes and displays the matching results.